

Hola DeepSeek

¡Hola! ¿Cómo estás? 😊

Me alegra mucho saludarte. Soy DeepSeek, tu asistente de IA, y estoy aquí para lo que necesites. ¿En qué puedo ayudarte hoy? Puedes preguntarme lo que sea, desde dudas cotidianas hasta temas más complejos, ¡estoy listo para conversar contigo!

Cuéntame, ¿qué te trae por aquí? 🌟



=====...
TXT 89.17KB

Soy estudiante de ingeniería de sistemas y el profesor nos facilitó el siguiente código de un proyecto monolítico desarrollado en PHP (Ver documento cargado). Él quiere que aprendamos pruebas Unitarias y de Integración. Ayúdame a entender: 1) Explicación de funcionamiento; 2) Configuración; 3) Pruebas Unitarias; 4) Pruebas de Integración. No avancemos hasta que cada punto me quede claro, utilizaré la palabra "siguiente" para avanzar.

¡Hola! Me alegra mucho que estés dando este importante paso en tu formación como ingeniero de sistemas. Las pruebas unitarias y de integración son habilidades fundamentales que te diferenciarán como profesional. Vamos a abordar este proyecto paso a paso, con la paciencia que necesitas.

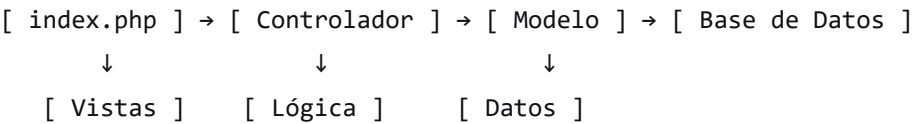
Comencemos con el **Punto 1: Explicación de funcionamiento**. Antes de hablar de pruebas, necesitas entender qué hace este código y cómo funciona.

📌 PUNTO 1: EXPLICACIÓN DE FUNCIONAMIENTO

Este es un sistema de inventario/ventas con control de usuarios y roles. Está construido bajo el patrón **MVC (Modelo-Vista-Controlador)**.

Arquitectura General:

text



Flujo de ejecución (index.php - el punto de entrada):

- 1. Inicia sesión y buffer de salida (para manejar redirecciones).
- 2. Determina el controlador a ejecutar:
 - Si no se pasa ?c=... por URL, carga Landing (página principal).
 - Si existe el archivo del controlador, lo incluye y crea una instancia.
- 3. Determina la acción a ejecutar:
 - Si no se pasa ?a=... , usa main como acción por defecto.
- 4. Verifica autenticación:
 - Landing y Login → muestran cabecera y pie de página públicos.
 - Si hay sesión activa (\$_SESSION['session']): carga el header/footer según el rol.
 - Si no hay sesión: redirige al inicio.

Controladores principales:

Controlador	Función
Landing	Muestra la página principal pública (tienda)
Login	Gestiona el inicio de sesión (GET muestra formulario, POST valida credenciales)
Dashboard	Muestra el panel según el rol del usuario
Users	CRUD de roles y usuarios (con permisos según rol)

Controlador	Función
Logout	Cierra la sesión

Modelo User (el más importante):

Características técnicas a destacar:

- **Sobrecarga de constructores:** usa `__construct0()` , `__construct2()` , `__construct8()` , `__construct9()` según la cantidad de parámetros.
- **Cifrado SHA1** para contraseñas (en `login()` , `create_user()` , `update_user()`).

Métodos clave:

Método	Función
<code>login()</code>	Busca usuario por email y contraseña, devuelve objeto User o false
<code>create_rol()</code> / <code>read_roles()</code> / <code>update_rol()</code> / <code>delete_rol()</code>	CRUD de roles
<code>create_user()</code> / <code>read_users()</code> / <code>update_user()</code> / <code>delete_user()</code>	CRUD de usuarios

Base de datos (estructura implícita):

Tabla ROLES:

text

rol_code (PK) | rol_name

Tabla USERS:

text

rol_code (FK) | user_code (PK) | user_name | user_lastname | user_id | user_email | user_pass | user_state

Vistas y roles:

- **Vistas públicas:** views/company/ (header, footer, index, login)
- **Vistas por rol:**
 - admin : acceso a todo (roles, usuarios, categorías, productos, ventas)
 - seller : acceso a usuarios y ventas
 - customer : acceso solo a compras y perfil

¿Qué hace cada acción? (Ejemplos con URL):

URL	Qué hace
?c=Login	Muestra formulario de login
?c=Dashboard	Muestra el panel según el rol
?c=Users&a=rolRead	Lista todos los roles (solo admin)
?c=Users&a=userCreate	Muestra formulario para crear usuario (admin/seller)
?c=Logout	Cierra sesión

Puntos débiles identificados (para pruebas):

1. Dependencia directa de la sesión en varios controladores.
2. No hay validación de entrada robusta (solo patterns en HTML).

3. Las contraseñas se pasan en texto plano antes de cifrar.
4. El modelo mezcla lógica de negocio y acceso a datos (no hay capa DAO/Repository).
5. No hay manejo de transacciones en las operaciones.
6. Las vistas incluyen lógica PHP (mezcla de responsabilidades).

✅ ¿Te queda claro este punto?

Si tienes dudas sobre el flujo, los controladores, o algún método específico, pregúntame antes de avanzar. Cuando estés listo, escribe "**siguiente**" y pasaremos al **Punto 2: Configuración** (cómo preparar el entorno para hacer pruebas).

Ayúde a 2) Configurar el entorno de pruebas con PHPUnit

Perfecto. Entendido. Vamos a profundizar en el **Punto 2: Configuración del entorno de pruebas con PHPUnit**.

Configurar correctamente el entorno es clave para que las pruebas sean confiables y no afecten tu proyecto real. Te guiaré paso a paso.

📌 PUNTO 2: CONFIGURACIÓN DEL ENTORNO DE PRUEBAS

2.1. Estructura de directorios recomendada

Primero, organiza tu proyecto para separar el código de producción de las pruebas:

text

```
mi-proyecto/
├── assets/
├── controllers/
├── models/
├── views/
├── tests/                ← NUEVA CARPETA
│   ├── unit/            ← Pruebas unitarias
│   │   └── UserTest.php
```

```
| | └─ DatabaseTest.php
| └─ integration/      ← Pruebas de integración
| | └─ LoginTest.php
| | └─ UserCRUDTest.php
| └─ bootstrap.php     ← Configuración inicial
└─ vendor/             ← Dependencias (composer)
└─ composer.json
└─ phpunit.xml         ← Configuración de PHPUnit
└─ index.php
```

2.2. Instalación de PHPUnit con Composer

Paso 1: Asegúrate de tener Composer instalado. Si no, descárgalo desde getcomposer.org.

Paso 2: Crea un archivo `composer.json` en la raíz de tu proyecto con este contenido:

json

```
{
  "require-dev": {
    "phpunit/phpunit": "^9.0"
  },
  "autoload": {
    "classmap": [
      "controllers/",
      "models/"
    ]
  }
}
```

Paso 3: Instala las dependencias ejecutando en la terminal:

bash

```
composer install
```

Esto creará la carpeta `vendor/` con PHPUnit y generará el autoloader.

2.3. Configuración de PHPUnit (phpunit.xml)

Crea un archivo `phpunit.xml` en la raíz:

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/9.3/phpunit.xsd"
        bootstrap="tests/bootstrap.php"
        colors="true"
        stopOnFailure="false">
  <testsuites>
    <testsuite name="Unit Tests">
      <directory>tests/unit</directory>
    </testsuite>
    <testsuite name="Integration Tests">
      <directory>tests/integration</directory>
    </testsuite>
  </testsuites>
</phpunit>
```

2.4. Archivo de arranque (tests/bootstrap.php)

Este archivo se ejecuta antes de cualquier prueba. Configura el autoloading y el entorno de pruebas:

php

```
<?php
// tests/bootstrap.php

require_once __DIR__ . '/../vendor/autoload.php';

// Si usas una base de datos de prueba, define constantes
define('DB_HOST', 'localhost');
define('DB_NAME', 'db_inventory_test'); // ← Base de datos para pruebas
define('DB_USER', 'root');
define('DB_PASS', '');

// Simula las sesiones para pruebas (importante)
```

```
$_SESSION = [];  
  
// Carga las clases principales  
require_once __DIR__ . '/../models/DataBase.php';  
require_once __DIR__ . '/../models/User.php';
```

2.5. Configuración de la Base de Datos para pruebas

Recomendación: Usa una base de datos separada para no dañar tus datos reales.

1. Crea la base de datos de prueba (desde phpMyAdmin o MySQL Workbench):

sql

```
CREATE DATABASE db_inventory_test;
```

2. Copia la estructura de tus tablas a esta nueva base:

sql

```
CREATE TABLE ROLES (  
    rol_code VARCHAR(10) PRIMARY KEY,  
    rol_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE USERS (  
    rol_code VARCHAR(10),  
    user_code VARCHAR(10) PRIMARY KEY,  
    user_name VARCHAR(50),  
    user_lastname VARCHAR(50),  
    user_id VARCHAR(20),  
    user_email VARCHAR(70) UNIQUE,  
    user_pass VARCHAR(200),  
    user_state TINYINT,  
    FOREIGN KEY (rol_code) REFERENCES ROLES(rol_code)  
);
```

3. Inserta datos de prueba iniciales:

sql


```

INSERT INTO ROLES VALUES ('R001', 'admin');
INSERT INTO ROLES VALUES ('R002', 'seller');
INSERT INTO ROLES VALUES ('R003', 'customer');

INSERT INTO USERS VALUES ('R001', 'U001', 'Admin', 'Sistema', '12345', 'admin@test.com', SHA1('admin123'), 1);
INSERT INTO USERS VALUES ('R002', 'U002', 'Vendedor', 'Prueba', '67890', 'seller@test.com', SHA1('seller123'), 1);

```

4. **Modifica temporalmente** DataBase.php **para que apunte a la base de prueba** (o usa variables de entorno).

2.6. Creación de helpers para pruebas

Para facilitar las pruebas, puedes crear una clase base con métodos útiles:

php

```

<?php
// tests/TestCase.php

use PHPUnit\Framework\TestCase;

abstract class MyTestCase extends TestCase
{
    protected function setUp(): void
    {
        parent::setUp();
        // Iniciar sesión simulada para cada prueba
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        $_SESSION = []; // Limpiar sesión antes de cada prueba
    }

    protected function tearDown(): void
    {
        // Limpiar sesión después de cada prueba
        $_SESSION = [];
        parent::tearDown();
    }
}

```

```

    }

    protected function createMockUser($rol = 'admin', $active = true)
    {
        $user = new User();
        $user->setRolCode($rol === 'admin' ? 'R001' : 'R002');
        $user->setRolName($rol);
        $user->setUserCode('U999');
        $user->setUserName('Test');
        $user->setUserLastName('User');
        $user->setUserId('99999');
        $user->setUserEmail('test@test.com');
        $user->setUserPass('test123');
        $user->setUserState($active ? 1 : 0);
        return $user;
    }
}

```

2.7. Verificación de la instalación

Crea un archivo de prueba simple para verificar que todo funciona:

php

```

<?php
// tests/unit/ExampleTest.php

use PHPUnit\Framework\TestCase;

class ExampleTest extends TestCase
{
    public function testBasicAssertion()
    {
        $this->assertTrue(true);
        $this->assertEquals(2, 1 + 1);
    }
}

```

Ejecuta desde la terminal:

```
bash
```

```
vendor/bin/phpunit
```

Si ves un mensaje verde tipo *"OK (1 test, 2 assertions)"*, ¡está todo listo! 🎉

2.8. Ajustes adicionales para pruebas con sesión y base de datos

Dado que tu proyecto usa `$_SESSION` y conexión a MySQL, considera estos tips:

Desafío	Solución
Sesiones en pruebas	Usa <code>session_start()</code> en el bootstrap y limpia con <code>session_destroy()</code> después de cada prueba
Dependencia de DB real	Crea una base de datos de prueba y usa transacciones para que los cambios no persistan
Métodos estáticos en <code>DataBase</code>	Puedes crear un método <code>setConnection()</code> para inyectar una conexión mock en las pruebas
<code>\$_SESSION['session']</code>	Simúlalo directamente: <code>\$_SESSION['session'] = 'admin';</code> en tus pruebas

2.9. Ejemplo de estructura de prueba de la base de datos

```
php
```

```
// tests/unit/DatabaseTest.php
```

```
use PHPUnit\Framework\TestCase;
```

```
class DatabaseTest extends TestCase
{
    private $pdo;

    protected function setUp(): void
    {
        parent::setUp();
        $this->pdo = DataBase::connection();
        // Iniciar transacción para no afectar la DB real
        $this->pdo->beginTransaction();
    }

    protected function tearDown(): void
    {
        // Revertir cambios después de cada prueba
        $this->pdo->rollBack();
        parent::tearDown();
    }

    public function testConnection()
    {
        $this->assertInstanceOf(PDO::class, $this->pdo);
    }
}
```

✓ Resumen de lo que hemos configurado:

1. **Composer** → gestiona dependencias y autoloading.
2. **phpunit.xml** → configura PHPUnit.
3. **tests/bootstrap.php** → prepara el entorno de pruebas.
4. **Base de datos de prueba** → separada de la producción.
5. **Clase base** MyTestCase → reutiliza lógica común.
6. **Manejo de sesiones y transacciones** → para pruebas seguras.

 ¿Te queda claro este punto?

Pregúntame si:

- No tienes claro cómo usar Composer.
- Tienes dudas con la base de datos de prueba.
- No sabes cómo ejecutar PHPUnit desde la terminal.
- Necesitas más detalles sobre el archivo `phpunit.xml`.

Cuando estés listo, escribe **"siguiente"** y avanzaremos al **Punto 3: Pruebas Unitarias** (cómo probar cada método del modelo `User` de forma aislada).

Intenté instalar composer pero no quedó en los programas instalados y al ejecutar en la terminal el comando "composer install", me sale lo siguiente: "composer : El término 'composer' no se reconoce como nombre de un cmdlet, función, archivo de script o programa ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta es correcta e inténtelo de nuevo.

En línea: 1 Carácter: 1

+ composer install

+ ~~~~~

+ CategoryInfo : ObjectNotFound: (composer:String) [], CommandNotFoundException

+ FullyQualifiedErrorId : CommandNotFoundException"; ayúdame con la solución de este problema y cuando lo haya solucionado, me vuelves a explicar lo siguiente.

Excelente pregunta. Este es un error muy común cuando recién empiezas con Composer. Vamos a solucionarlo paso a paso.

SOLUCIÓN: Composer no está instalado o no está en el PATH

El error indica que Windows no encuentra el ejecutable `composer` en la terminal. Tienes dos opciones:

OPCIÓN 1: Instalar Composer correctamente (Recomendado)

1. Descarga el instalador oficial:

- Ve a: <https://getcomposer.org/download/>
- Haz clic en "**Composer-Setup.exe**" (la versión más reciente)

2. Ejecuta el instalador:

- Acepta los términos.
- En la pantalla de selección de PHP, asegúrate de que apunte a tu instalación de PHP (ej: C:\xampp\php\php.exe si usas XAMPP).
- **IMPORTANTE:** Marca la opción "**Add to PATH**" (agregar al PATH del sistema) cuando aparezca.
- Completa la instalación.

3. Reinicia tu terminal (cierra y vuelve a abrir CMD/PowerShell).

4. Verifica la instalación:

```
bash
```

```
composer --version
```

Deberías ver algo como: Composer version 2.x.x

OPCIÓN 2: Usar Composer sin instalación global (alternativa)

Si no quieres instalarlo globalmente, puedes usar el archivo `composer.phar` :

1. Descarga el archivo PHAR:

- Ve a: <https://getcomposer.org/composer.phar>
- O desde terminal:

```
bash
```

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
php -r "unlink('composer-setup.php');"
```

2. Ejecuta comandos con:

```
bash
```

```
php composer.phar install
```

En lugar de `composer install`

OPCIÓN 3: Si ya está instalado pero no en el PATH

Composer podría estar instalado pero no reconocido. Sigue estos pasos:

1. Encuentra dónde está Composer:

- Normalmente en: `C:\ProgramData\ComposerSetup\bin\composer`
- O en: `C:\Users\TU_USUARIO\AppData\Roaming\Composer\vendor\bin\composer`

2. Agrégalo al PATH manualmente:

- Presiona `Win + R`, escribe `sysdm.cpl` y presiona Enter.
- Ve a la pestaña "**Opciones avanzadas**" → "**Variables de entorno**".
- En "**Variables del sistema**", busca la variable `Path` y haz clic en "**Editar**".
- Agrega la ruta donde está el ejecutable de Composer (ej:
`C:\ProgramData\ComposerSetup\bin`).
- Haz clic en "**Aceptar**" y reinicia la terminal.



Verificación final

Después de cualquiera de las opciones, prueba esto en la terminal:

```
bash
```

```
# Ver si Composer está disponible  
composer --version
```

```
# Si no funciona, prueba con PHP directamente
```

```
php -v
php composer.phar --version
```

💡 Alternativa: Usar el instalador manual de PHPUnit

Si Composer sigue dando problemas, puedes instalar PHPUnit directamente:

1. Descarga PHPUnit PHAR:

```
bash

php -r "copy('https://phar.phpunit.de/phpunit-9.phar', 'phpunit.phar');"
```

2. Ejecuta pruebas con:

```
bash

php phpunit.phar --configuration phpunit.xml
```

📋 Plan de acción recomendado para ti:

1. Verifica si tienes PHP instalado:

```
bash

php -v
```

Si no ves versión, instala XAMPP o WAMP.

2. Instala Composer con el instalador (Opción 1) y marca "Add to PATH".

3. Reinicia la terminal y prueba `composer --version`.

4. Si todo funciona, ve a tu proyecto y ejecuta:

```
bash

composer install
```


5. Si no funciona, usa la Opción 2 (archivo PHAR).

✅ ¿Qué hacer después de solucionarlo?

Cuando ya puedas ejecutar `composer install` sin errores:

1. Se creará la carpeta `vendor/` con PHPUnit.
2. Podrás ejecutar `vendor/bin/phpunit` para correr pruebas.
3. Podremos continuar con el **Punto 3: Pruebas Unitarias**.

❓ Dime en qué punto estás:

- A) Ya instalé Composer y funcionó `composer --version`.
- B) Instalé Composer pero sigue sin funcionar.
- C) Prefiero usar la alternativa sin instalación global.
- D) Necesito ayuda para verificar mi instalación de PHP.

Cuando hayas solucionado el problema y puedas ejecutar `composer install` sin errores, escribe "siguiente" y continuaremos con el Punto 3. Si necesitas más ayuda con la instalación, solo dímelo.

Ya solucioné lo de "composer", ahora ayúdame a 2) Configurar el entorno de pruebas con PHPUnit

¡Excelente! Me alegra que hayas solucionado el problema con Composer. Ahora vamos a profundizar en la configuración completa del entorno de pruebas. Como ya vimos la teoría, ahora vamos a la práctica paso a paso.

📌 PUNTO 2: CONFIGURACIÓN DEL ENTORNO DE PRUEBAS CON PHPUNIT (PRÁCTICO)

Paso 1: Crear la estructura de carpetas

Desde la raíz de tu proyecto, crea las siguientes carpetas:

```
bash

tests/
tests/unit/
tests/integration/
```

Puedes hacerlo desde el explorador de archivos o desde la terminal:

```
bash

mkdir tests
mkdir tests\unit
mkdir tests\integration
```

Paso 2: Configurar composer.json

Verifica que tu archivo `composer.json` tenga esta estructura. Si no lo tienes, créalo en la raíz del proyecto:

```
json

{
    "require-dev": {
        "phpunit/phpunit": "^9.0"
    },
    "autoload": {
        "classmap": [
            "controllers/",
            "models/"
        ]
    }
}
```

Luego ejecuta para instalar PHPUnit:

```
bash
```

```
composer install
```

Esto creará la carpeta `vendor/` con todas las dependencias.

Paso 3: Crear el archivo de configuración phpunit.xml

Crema un archivo llamado `phpunit.xml` en la raíz de tu proyecto con este contenido:

```
xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/9.3/phpunit.xsd"
    bootstrap="tests/bootstrap.php"
    colors="true"
    stopOnFailure="false"
    verbose="true">
  <testsuites>
    <testsuite name="Unit Tests">
      <directory>tests/unit</directory>
    </testsuite>
    <testsuite name="Integration Tests">
      <directory>tests/integration</directory>
    </testsuite>
  </testsuites>
  <php>
    <env name="APP_ENV" value="testing"/>
    <env name="DB_HOST" value="localhost"/>
    <env name="DB_NAME" value="db_inventory_test"/>
    <env name="DB_USER" value="root"/>
    <env name="DB_PASS" value=""/>
  </php>
</phpunit>
```

Paso 4: Crear el archivo de arranque tests/bootstrap.php

Crea tests/bootstrap.php con este código:

php

```
<?php
// tests/bootstrap.php

// Cargar el autoloader de Composer
require_once __DIR__ . '/../vendor/autoload.php';

// Configurar variables de entorno para pruebas
define('DB_HOST', 'localhost');
define('DB_NAME', 'db_inventory_test');
define('DB_USER', 'root');
define('DB_PASS', '');

// Cargar las clases del proyecto
require_once __DIR__ . '/../models/DataBase.php';
require_once __DIR__ . '/../models/User.php';

// Iniciar sesión para las pruebas (simular)
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// Limpiar sesión antes de cada prueba
$_SESSION = [];

// Función de ayuda para resetear la base de datos
function resetTestDatabase() {
    try {
        $pdo = DataBase::connection();
        // Eliminar datos existentes en orden correcto (primero users, luego roles)
        $pdo->exec("DELETE FROM USERS");
        $pdo->exec("DELETE FROM ROLES");

        // Insertar datos de prueba
        $pdo->exec("INSERT INTO ROLES VALUES ('R001', 'admin')");
        $pdo->exec("INSERT INTO ROLES VALUES ('R002', 'seller')");
        $pdo->exec("INSERT INTO ROLES VALUES ('R003', 'customer')");

        $pdo->exec("INSERT INTO USERS VALUES ('R001', 'U001', 'Admin', 'Sistema', '1234
5', 'admin@test.com', '' . sha1('admin123') . '', 1)");
        $pdo->exec("INSERT INTO USERS VALUES ('R002', 'U002', 'Vendedor', 'Prueba', '6789
```

```

0', 'seller@test.com', '"' . sha1('seller123') . '", 1)");
    $pdo->exec("INSERT INTO USERS VALUES ('R003', 'U003', 'Cliente', 'Test', '11111',
'customer@test.com', '"' . sha1('customer123') . '", 1)");

    return true;
} catch (Exception $e) {
    echo "Error al resetear la base de datos: " . $e->getMessage();
    return false;
}
}

```

Paso 5: Crear una clase base para pruebas

Crea tests/TestCase.php (clase base que extenderán todas tus pruebas):

php

```

<?php
// tests/TestCase.php

use PHPUnit\Framework\TestCase as BaseTestCase;

abstract class TestCase extends BaseTestCase
{
    protected $pdo;

    protected function setUp(): void
    {
        parent::setUp();

        // Iniciar sesión para pruebas
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        $_SESSION = [];

        // Obtener conexión a la base de datos de prueba
        $this->pdo = DataBase::connection();
        // Iniciar transacción para no afectar la DB real
        $this->pdo->beginTransaction();
    }
}

```

```

protected function tearDown(): void
{
    // Revertir todos los cambios después de cada prueba
    if ($this->pdo && $this->pdo->inTransaction()) {
        $this->pdo->rollBack();
    }
    $_SESSION = [];
    parent::tearDown();
}

// Helper para crear un usuario de prueba
protected function createTestUser($rol = 'admin', $active = true)
{
    $user = new User();
    $user->setRolCode($rol === 'admin' ? 'R001' : ($rol === 'seller' ? 'R002' : 'R00
3'));
    $user->setRolName($rol);
    $user->setUserCode('TEST' . rand(100, 999));
    $user->setUserName('Test');
    $user->setUserLastName('User');
    $user->setUserId('99999');
    $user->setUserEmail('test' . rand(100, 999) . '@test.com');
    $user->setUserPass('test123');
    $user->setUserState($active ? 1 : 0);
    return $user;
}

// Helper para simular sesión de usuario
protected function loginAs($rol = 'admin')
{
    $user = new User();
    $user->setRolCode($rol === 'admin' ? 'R001' : ($rol === 'seller' ? 'R002' : 'R00
3'));
    $user->setRolName($rol);
    $user->setUserCode('U001');
    $user->setUserName('Test');
    $user->setUserLastName('User');
    $user->setUserId('12345');
    $user->setUserEmail($rol . '@test.com');
    $user->setUserPass(sha1($rol . '123'));
    $user->setUserState(1);

    $_SESSION['session'] = $rol;
}

```

```

        $_SESSION['profile'] = serialize($user);

        return $user;
    }
}

```

Paso 6: Crear una prueba de verificación

Crea `tests/unit/ExampleTest.php` para verificar que todo funciona:

php

```

<?php
// tests/unit/ExampleTest.php

require_once __DIR__ . '/../TestCase.php';

class ExampleTest extends TestCase
{
    public function testDatabaseConnection()
    {
        $this->assertInstanceOf(PDO::class, $this->pdo);
    }

    public function testBasicAssertions()
    {
        $this->assertTrue(true);
        $this->assertEquals(2, 1 + 1);
        $this->assertNotEmpty($_SESSION);
        $_SESSION['test'] = 'ok';
        $this->assertEquals('ok', $_SESSION['test']);
    }

    public function testResetDatabase()
    {
        // Verificar que la función de reset funciona
        $result = resetTestDatabase();
        $this->assertTrue($result);

        // Verificar que hay datos en la base
        $stmt = $this->pdo->query("SELECT COUNT(*) FROM ROLES");
    }
}

```

```

        $count = $stmt->fetchColumn();
        $this->assertEquals(3, $count);
    }
}

```

Paso 7: Modificar DataBase.php para pruebas

Para que las pruebas usen la base de datos de prueba sin modificar el código original, crea una versión modificada temporalmente:

Opción A (Recomendada): Modifica `models/DataBase.php` para usar variables de entorno:

php

```

<?php
class DataBase{
    public static function connection(){
        $hostname = getenv('DB_HOST') ?: "localhost";
        $port = "3306";
        $database = getenv('DB_NAME') ?: "db_inventory";
        $username = getenv('DB_USER') ?: "root";
        $password = getenv('DB_PASS') ?: "";

        try {
            $pdo = new PDO("mysql:host=$hostname;port=$port;dbname=$database;charset=utf
8", $username, $password);
            $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
            return $pdo;
        } catch (PDOException $e) {
            die("Error de conexión: " . $e->getMessage());
        }
    }
}

```

Opción B: Crear una clase `TestDatabase` para pruebas que extienda de `DataBase`.

Paso 8: Crear la base de datos de prueba

Ejecuta este script SQL en phpMyAdmin o desde la terminal:

```
sql

-- Crear la base de datos de prueba
CREATE DATABASE IF NOT EXISTS db_inventory_test;
USE db_inventory_test;

-- Crear tablas
CREATE TABLE IF NOT EXISTS ROLES (
    rol_code VARCHAR(10) PRIMARY KEY,
    rol_name VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS USERS (
    rol_code VARCHAR(10),
    user_code VARCHAR(10) PRIMARY KEY,
    user_name VARCHAR(50),
    user_lastname VARCHAR(50),
    user_id VARCHAR(20),
    user_email VARCHAR(70) UNIQUE,
    user_pass VARCHAR(200),
    user_state TINYINT,
    FOREIGN KEY (rol_code) REFERENCES ROLES(rol_code)
);

-- Insertar datos iniciales
INSERT IGNORE INTO ROLES VALUES ('R001', 'admin');
INSERT IGNORE INTO ROLES VALUES ('R002', 'seller');
INSERT IGNORE INTO ROLES VALUES ('R003', 'customer');

INSERT IGNORE INTO USERS VALUES ('R001', 'U001', 'Admin', 'Sistema', '12345', 'admin@test.com', SHA1('admin123'), 1);
INSERT IGNORE INTO USERS VALUES ('R002', 'U002', 'Vendedor', 'Prueba', '67890', 'seller@test.com', SHA1('seller123'), 1);
INSERT IGNORE INTO USERS VALUES ('R003', 'U003', 'Cliente', 'Test', '11111', 'customer@test.com', SHA1('customer123'), 1);
```

Paso 9: Ejecutar las pruebas

Desde la terminal, en la raíz de tu proyecto:

```
bash
```

```
# Ejecutar todas las pruebas
```

```
vendor/bin/phpunit
```

```
# Ejecutar solo pruebas unitarias
```

```
vendor/bin/phpunit --testsuite "Unit Tests"
```

```
# Ejecutar una prueba específica
```

```
vendor/bin/phpunit tests/unit/ExampleTest.php
```

```
# Ver más detalles
```

```
vendor/bin/phpunit --verbose
```

Paso 10: Verificar que todo funciona correctamente

Deberías ver una salida similar a:

```
text
```

```
PHPUnit 9.6.x by Sebastian Bergmann and contributors.
```

```
...
```

```
3 / 3 (100%)
```

```
Time: 00:00.123, Memory: 6.00 MB
```

```
OK (3 tests, 5 assertions)
```

Lista de verificación final

Confirma que tienes:

- ☐ Carpeta `tests/` con subcarpetas `unit/` e `integration/`
- ☐ Archivo `composer.json` con PHPUnit configurado
- ☐ Archivo `phpunit.xml` en la raíz
- ☐ Archivo `tests/bootstrap.php` creado

- Archivo tests/TestCase.php creado
- Base de datos db_inventory_test creada
- DataBase.php modificado para usar variables de entorno
- Prueba ExampleTest.php funcionando

¿Te ha quedado claro este punto?

Si tienes dudas sobre:

- La estructura de carpetas
- El contenido de los archivos de configuración
- Cómo crear la base de datos de prueba
- Cómo ejecutar los comandos

Cuando hayas configurado todo y la prueba de ejemplo funcione correctamente, escribe "siguiente" y pasaremos al Punto 3: Pruebas Unitarias.

```
Este código en qué archivo va? "<?php
// tests/TestCase.php

use PHPUnit\Framework\TestCase;

abstract class MyTestCase extends TestCase
{
    protected function setUp(): void
    {
        parent::setUp();
        // Iniciar sesión simulada para cada prueba
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        $_SESSION = []; // Limpiar sesión antes de cada prueba
    }

    protected function tearDown(): void
    {

```

```

        // Limpiar sesión después de cada prueba
        $_SESSION = [];
        parent::tearDown();
    }

    protected function createMockUser($rol = 'admin', $active = true)
    {
        $user = new User();
        $user->setRolCode($rol === 'admin' ? 'R001' : 'R002');
        $user->setRolName($rol);
        $user->setUserCode('U999');
        $user->setUserName('Test');
        $user->setUserLastName('User');
        $user->setUserId('99999');
        $user->setUserEmail('test@test.com');
        $user->setUserPass('test123');
        $user->setUserState($active ? 1 : 0);
        return $user;
    }
}

```

Perfecta pregunta. Te voy a aclarar exactamente dónde va cada archivo y por qué.

UBICACIÓN CORRECTA DE CADA ARCHIVO

El código que me muestras es una **clase base abstracta** que servirá como "plantilla" para todas tus pruebas. Debe ir en:

text

tests/TestCase.php

Pero con un detalle importante: el nombre de la clase debe ser `TestCase` (no `MyTestCase`) para que sea fácil de extender.



VERSIÓN CORREGIDA DEL ARCHIVO

Crea el archivo `tests/TestCase.php` con este contenido (he ajustado el nombre de la clase y mejorado algunos detalles):

php

```
<?php
// tests/TestCase.php

use PHPUnit\Framework\TestCase as BaseTestCase;

abstract class TestCase extends BaseTestCase
{
    protected function setUp(): void
    {
        parent::setUp();

        // Iniciar sesión simulada para cada prueba
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        $_SESSION = []; // Limpiar sesión antes de cada prueba
    }

    protected function tearDown(): void
    {
        // Limpiar sesión después de cada prueba
        $_SESSION = [];
        parent::tearDown();
    }

    protected function createMockUser($rol = 'admin', $active = true)
    {
        $user = new User();
        $user->setRolCode($rol === 'admin' ? 'R001' : 'R002');
        $user->setRolName($rol);
        $user->setUserCode('U999');
        $user->setUserName('Test');
        $user->setUserLastName('User');
        $user->setUserId('99999');
        $user->setUserEmail('test@test.com');
        $user->setUserPass('test123');
        $user->setUserState($active ? 1 : 0);
        return $user;
    }
}
```

```
}  
}
```

ESTRUCTURA FINAL DE CARPETAS

Así debería quedar tu proyecto:

text

```
mi-proyecto/  
|  
├─ assets/           (tus assets existentes)  
├─ controllers/      (tus controladores existentes)  
|   ├─ Dashboard.php  
|   ├─ Landing.php  
|   ├─ Login.php  
|   ├─ Logout.php  
|   └─ Users.php  
|  
├─ models/           (tus modelos existentes)  
|   ├─ DataBase.php  
|   └─ User.php  
|  
├─ views/            (tus vistas existentes)  
|   ├─ company/  
|   ├─ modules/  
|   └─ roles/  
|  
├─ tests/            ← NUEVA CARPETA (creada por ti)  
|   ├─ unit/          ← NUEVA CARPETA  
|   |   └─ (aquí irán las pruebas unitarias)  
|   ├─ integration/   ← NUEVA CARPETA  
|   |   └─ (aquí irán las pruebas de integración)  
|   ├─ bootstrap.php  ← NUEVO ARCHIVO  
|   └─ TestCase.php   ← NUEVO ARCHIVO (ESTE ES EL QUE PREGUNTAS)  
|  
├─ vendor/           (creado por Composer)  
├─ composer.json      (tu archivo de configuración)  
├─ phpunit.xml         (configuración de PHPUnit)  
└─ index.php           (tu punto de entrada)
```

💡 EXPLICACIÓN DE POR QUÉ VA AHÍ

1. tests/ es la carpeta estándar para pruebas en PHP.
2. TestCase.php es una clase base que **NO** se ejecuta directamente como prueba, sino que es extendida por otras clases de prueba.
3. Al estar en tests/ , puede ser cargada por el autoloader o incluida manualmente en bootstrap.php .

🔗 CÓMO SE USA ESTE ARCHIVO

Luego, en tus pruebas específicas (ej: tests/unit/UserTest.php), harás:

php

```
<?php
```

```
// tests/unit/UserTest.php
```

```
require_once __DIR__ . '/../TestCase.php';
```

```
class UserTest extends TestCase // ← Extiende de tu clase base
```

```
{
```

```
    public function testCreateUser()
```

```
    {
```

```
        // Usas el helper que definiste en TestCase
```

```
        $user = $this->createMockUser('admin', true);
```

```
        $this->assertEquals('admin', $user->getRolName());
```

```
        $this->assertEquals(1, $user->getUserState());
```

```
    }
```

```
}
```

⚠️ **IMPORTANTE:** Asegúrate de que el autoloader cargue los modelos

Para que `new User()` funcione dentro de `TestCase.php`, necesitas que el autoloader de Composer cargue la clase `User`. Verifica que en `composer.json` tengas:

```
json

{
    "require-dev": {
        "phpunit/phpunit": "^9.0"
    },
    "autoload": {
        "classmap": [
            "controllers/",
            "models/"
        ]
    }
}
```

Luego ejecuta:

```
bash

composer dump-autoload
```

RESUMEN RÁPIDO

Archivo	Ubicación	Propósito
TestCase.php	tests/TestCase.php	Clase base para todas las pruebas
bootstrap.php	tests/bootstrap.php	Configuración inicial del entorno
UserTest.php	tests/unit/UserTest.php	Pruebas unitarias del modelo User
LoginTest.php	tests/integration/LoginTest.php	Pruebas de integración del login

 ¿Te quedó claro?

Si tienes dudas sobre:

- ¿Por qué es abstracta?
- ¿Cómo se extiende?
- ¿Qué pasa si no existe la clase User?

Cuando hayas creado el archivo en la ubicación correcta y entiendas su propósito, escribe "siguiente" y pasaremos a crear pruebas unitarias reales.